

# Midterm 1

M 375T Mathematical Foundations of Machine Learning — Fall 2026

Tuesday, October 6 · 75 minutes · closed book · no devices

Name: \_\_\_\_\_ UT EID: \_\_\_\_\_

---

## Instructions.

- There are 4 questions worth 25 points each. The total is 100 points.
  - Every question starts with a short NumPy program. The program is not run for you. Read it, then answer the parts. “Predict the output” means: write what the `print` statement would show.
  - Write every answer inside the box below the question. If you need more room, use the back of the page and write “see back” in the box.
  - Show the computation that produces each number. A correct number with no computation earns at most half credit.
  - Conventions of the course: `W.sum(axis=1)` is the degree vector  $d$ ;  $P_{ij} = W_{ij}/d_i$ , so the rows of  $P$  sum to one; laws are column vectors and move by  $\mu_{t+1} = P^\top \mu_t$  (`P.T @ mu`); the lazy walk is  $Q = \frac{1}{2}(I + P)$ ; the Laplacian is  $L = D - W$ . `np.linalg.eigvalsh(A)` returns the eigenvalues of a symmetric matrix  $A$  in increasing order.
  - Leave fractions as fractions. You do not need decimals unless the question asks for them.
- 

**Do not turn the page until you are told to start.**

1. Ordinary and lazy walks, in the long run.

[25 pts]

```
import numpy as np
W = np.array([[0, 1, 1],
              [1, 0, 0],
              [1, 0, 0]], dtype=float)
d = W.sum(axis=1)
P = W / d[:, None]
Q = (np.eye(3) + P) / 2

mu = np.array([0., 1., 0.])
for t in range(100):
    mu = P.T @ mu
print(np.round(mu, 3))          # line 1

nu = np.array([0., 1., 0.])
for t in range(100):
    nu = Q.T @ nu
print(np.round(nu, 3))          # line 2
```

You may use that  $w_1 = (2, 1, 1)^T$ ,  $w_2 = (2, -1, -1)^T$ ,  $w_3 = (0, 1, -1)^T$  are eigenvectors of  $P^T$ .

- (a) Draw the graph. Write  $d$  and  $P$ . Find the eigenvalues of  $P$ . [7 pts]
- (b) Predict line 1. Show your work. [10 pts]
- (c) Predict line 2. Show your work. [8 pts]

Answer:

2. The best Gaussian, mean and variance.

[25 pts]

```
x = np.array([1., 2., 6., 7.])

def H(m, s2):
    return np.mean(0.5*np.log(2*np.pi*s2) + (x - m)**2 / (2*s2))

print(x.mean(), x.var())           # line 1
print(H(4.0, 6.5) < H(4.0, 4.0))   # line 2
print(H(4.0, 6.5) < H(3.0, 6.5))   # line 3
```

- (a) What does  $H(m, s^2)$  compute? Write it as a formula in  $m$ ,  $s^2$  and the data. [5 pts]
- (b) Predict line 1. [3 pts]
- (c) Predict line 3. Justify it: compute  $\partial H / \partial m$  and show that for fixed  $s^2$  the minimizer is  $\bar{x}$ . [7 pts]
- (d) Predict line 2. Justify it: for  $m = \bar{x}$ , compute  $\partial H / \partial s^2$  and show that the minimizer is  $s^2 = \hat{\sigma}^2 = \frac{1}{n} \sum_i (x_i - \bar{x})^2$ . [10 pts]

Answer:

3. **The energy of a cut.** The code builds the weight matrix of an undirected graph on the vertices  $0, 1, 2, 3$ . [25 pts]

```
import numpy as np
W = np.array([[0, 3, 0, 1],
              [3, 0, 1, 0],
              [0, 1, 0, 2],
              [1, 0, 2, 0]], dtype=float)
d = W.sum(axis=1)
L = np.diag(d) - W

v = np.array([1., 1., -1., -1.])
print(v @ L @ v)           # line 1
print(L @ v)               # line 2
```

- (a) Draw the graph with its edge weights. Write  $d$  and  $L$ . [5 pts]
- (b) Predict line 1 using the Dirichlet form  $v^T L v = \frac{1}{2} \sum_{i,j} W_{ij} (v_i - v_j)^2$ . Which edges contribute, and how much each? [5 pts]
- (c) Predict line 2. Is  $v$  an eigenvector of  $L$ ? If so, give its eigenvalue. [8 pts]
- (d)  $v$  splits the vertices into  $A = \{0, 1\}$  and  $B = \{2, 3\}$ . Compute  $\text{cut}(A, B)$ ,  $\text{vol}(A)$ ,  $\text{vol}(B)$  and  $\text{Ncut}(A, B) = \text{cut}(A, B) \left( \frac{1}{\text{vol}(A)} + \frac{1}{\text{vol}(B)} \right)$ . How is  $\text{cut}(A, B)$  related to line 1? [7 pts]

Answer:

4. **Catching the impostor.** The model is  $N(0, 1)$ , and  $c = 0.6745$  is the number with  $\mathbb{P}(|Z| < c) = \frac{1}{2}$  for  $Z \sim N(0, 1)$ . You may use  $\sqrt{3} \approx 1.732$ . [25 pts]

```
rng = np.random.default_rng(0)
X = rng.uniform(-np.sqrt(3), np.sqrt(3), size=(50_000, 400))
k = (np.abs(X) < 0.6745).sum(axis=1)
print(k.shape)                # line 1
print(k.mean() / 400)         # line 2
print(k.std())                # line 3
eps = k / 400 - 0.5
D = 2 * eps**2
print(np.mean(2 * 400 * D > 3.84)) # line 4
```

- (a) Predict line 1. Say in one sentence what one entry of  $\mathbf{k}$  counts. [5 pts]
- (b) Predict line 2 (the value it is close to). Justify it with the probability that one uniform sample lands in  $(-c, c)$ . [8 pts]
- (c) Predict line 3 (the value it is close to). Name the distribution of one entry of  $\mathbf{k}$  and use its variance. [5 pts]
- (d) Predict line 4 (the value it is close to). Justify it. *Hint: compute  $2nD$  for a typical row, using (b).* [7 pts]

Answer:

---

Total: 100 points. Midterm 1 counts 20% of the course grade. If your final exam percentage is higher than your lower midterm percentage, the final replaces that midterm.